Triage de alertas SOC sobre 2M de eventos con modelos tabulares, de texto y de grafo

Propuesta de proyecto — Aprendizaje Automático (Primera entrega)

Author Name

8va sesión · 15 min + 5 min preguntas

Índice

¿Qué es un SOC? (1/3)

Un **Centro de Operaciones de Seguridad (SOC)** es el equipo y la plataforma que vigilan la infraestructura de una organización: servidores, estaciones de trabajo, red, nube y aplicaciones.

- No vigila un solo programa, sino **decenas de productos**: firewalls, antivirus/EDR, directorio de identidad (Active Directory), nube (AWS), virtualización, correo, DNS.
- Cada producto escribe **registros (logs)** continuamente: syslog, auditoría de Windows, flujos de red, telemetría de endpoint, autenticaciones.
- Un SOC mediano genera **millones de registros por semana**.

En nuestro caso: **2,011,674 eventos en solo 6 días** (26 jul – 01 ago 2024), de **4 organizaciones**, **19 productos** y **12 fabricantes**.

¿De dónde sale el etiquetado? (2/3)

Los registros no llegan etiquetados. Una plataforma de correlación —**WitFoo Precinct** v6.x— los normaliza, los sanea y les aplica un catálogo de **261 reglas de detección (lead rules)**.

- Si una regla se cumple → el evento queda marcado.
- Si varias señales relacionadas se agrupan → se levanta un **incidente**.

Un incidente contiene:

- las **señales disparadoras** (qué eventos lo provocaron),
- un **subgrafo de proveniencia** (qué entidad tocó a qué entidad),
- un **reporte de caza** en lenguaje natural.

Dato clave: las etiquetas son **correlación automática**, no veredicto de un analista humano.

Niveles y el problema del desbalance (3/3)

Cada evento cae en uno de tres niveles:

Nivel	Eventos	Significado
benign	1,897,153 (94.31 %)	ruido normal
suspicious	106,793 (5.31 %)	algo raro, vigilar
malicious	7,728 (0.38 %)	parte de un ataque (1,584 incidentes)

Razón de desbalance $\approx$ **245:1**. Consecuencia: un clasificador trivial que siempre dice "benigno" acierta el **94.31 %** — por eso elaccuracy no sirve y usaremos F1-macro y PR-AUC.

Motivación: por qué importa

- **Volumen**: 2M de eventos en 6 días; sin priorización automática, el analista gasta su tiempo en ruido.
- **Desbalance**: 0.38 % malicioso; el accuracy miente.
- **Asimetría de costos**: un falso negativo (brecha) es órdenes de magnitud peor que un falso positivo.
- **Contexto**: decidir exige mensaje + MITRE + cadena, no un número.
- **Etiquetas imperfectas**: 2,005,877 de 2,011,674 señales están `Unprocessed` (correlacionadas, nunca analizadas).
- **Deriva temporal**: malicious termina el 30 jul, benigno sigue al 01 ago $\rightarrow$ el split debe ser temporal.

Pregunta de investigación

Objetivo: estimar el nivel de riesgo de un evento con probabilidades calibradas y recomendación operativa (`escalar` / `monitorear`).

> *¿Es posible recuperar eventos maliciosos con desbalance 245:1 y deriva temporal combinando representaciones tabulares, textuales y estructurales, y qué compromiso existe con el nivel intermedio?*

Específicos: (i) caracterizar datos y leakage; (ii) $\geq$5 preprocesos; (iii) 3 modelos con complejidad; (iv) desplegar servicio + consola.

Panorama de la literatura

Tres familias revisadas:

- **Grafos de proveniencia**: NODLINK (APT en línea), KnowHow (CTI interpretable), FLASH (GNN temporal), threaTrace/GraphSAGE (nivel de nodo).
- **Mapeo a ATT&CK desde texto**: TRAM (MITRE, + notebooks SciBERT), comparativas TF-IDF vs transformers.
- **Desbalance**: Focal Loss; formulaciones sensibles al costo (FN $>>$ FP).

Dos casos se analizan en detalle: uno por **estructura** (grafo) y otro por **texto**.

Caso 1: Devign (NeurIPS 2019) — grafo

Qué hace: una red neuronal de grafos que aprende sobre representaciones del código en forma de grafo.

- Representación compuesta: AST como esqueleto + dependencias de control y de datos como aristas.
- Modelo: GGNN + módulo **Conv** que selecciona jerárquicamente nodos relevantes.
- **Tarea**: **clasificación binaria a nivel de función** — ¿es vulnerable o no?

Datos: etiquetados **manualmente** (∼600 horas-persona) de **4 librerías populares en C**: Linux Kernel, QEMU, Wireshark, FFmpeg — código real complejo, no sintético.
Resultado: supera a los baselines en promedio **+10.51 % accuracy y +8.68 % F1**; +27.99 % F1 vs analizadores estáticos; 74.11 % en 40 CVEs nuevos.

Caso 2: CVSS-BERT (ICMLA 2021) — texto

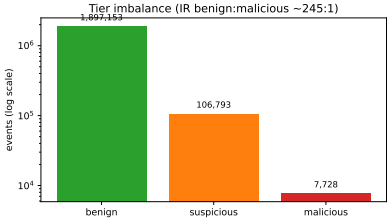
Qué hace: trabaja con **texto, no con código**. Entrena **varios clasificadores BERT, uno por cada métrica del vector CVSS**, a partir de la **descripción textual** de la vulnerabilidad.
Motivación: cuando se publica una vulnerabilidad, **solo hay una descripción**, y el análisis experto (vector CVSS) puede tardar días con miles de CVEs diarios.
Datos: 45,926 CVEs (2018–2020) de NVD. **Resultado**: accuracy 83.79–96.07 % por métrica; la severidad calculada queda **muy cerca del experto** (MAE 0.73; 55.3 % coincidencia exacta). Incluye saliency para explicar.
Limitación: estudios posteriores muestran que **los puntajes CVSS varían según quién los asigna** (NVD frente a otras fuentes) $\rightarrow$ las etiquetas dependen del contexto.

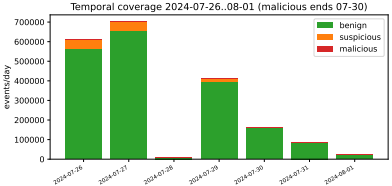
Síntesis y brecha

	Devign	CVSS-BERT	Propuesto
Entrada	grafo código	texto CVE	tabular + texto + subgrafo
Tarea	binaria	8 métricas $\rightarrow$ puntaje	3 niveles + ATT&CK
Etiquetas	manuales 600 h	NVD experto	Precinct automático
Señal	+10.51 % acc	MAE 0.73	cascada + compromiso

Brecha: la literatura opera sobre **código fuente** o **descripciones de vulnerabilidades**, pero no sobre **telemetría de operación en tiempo real** con desbalance extremo, deriva temporal y humano en el bucle.

EDA: desbalance y cobertura temporal





Desbalance (escala log): benign domina por órdenes de magnitud. **Temporal**: los leads maliciosos **terminan el 30-07**, el tráfico benigno continúa al 01-08 → entrenar con shuffle haría trampa; el split será **temporal**.

EDA: desbalance por organización y origen del ruido

- **ORG-0004** concentra 995,985 eventos (49.5 %) y 4,176 maliciosos; **ORG-0001** aporta solo 4,968 $\rightarrow$ sesgo declarado.
- Top productos: **AWS Instance Backup** (543k) y **Windows Active Directory** (421k) explican la mayor parte del volumen: el SOC vive filtrando respaldos y auditoría.
- `firewall_action`, lo que mejor entiende un modelo tabular, es minoría frente a la auditoría.

EDA: calidad, faltantes y fuga de información

- `severity` **49 % vacía**: no se imputa con mediana; se trata como **nivel propio** (string vacío). Hay 991k eventos sin severidad.
- **Fuga (leakage)**: `attack_techniques`, `mo_name`, `suspicion_score`, `matched_rules` están **vacíos en benign** → son priors del motor, **no deben usarse como features**.
- `label_confidence` se deriva del nivel → excluida.
- Duplicados por `artifact_id`: ninguno (clave única).

EDA: hipótesis que guían los modelos

Tres hipótesis verificadas en la exploración:

- **H1**: los tokens de explotación (`T1190`, `exploit`) separan malicious $\rightarrow$ justifica el **modelo de texto**.
- **H2**: los vecinos en ± 5 min (bloqueos, fallos de login) delatan movimiento lateral invisible en una fila $\rightarrow$ justifica el **modelo de grafo**.
- **H3**: el nivel **suspicious** sí es recuperable con campos tabulares $\rightarrow$ el modelo tabular sirve como base.

También se verificó que ATT&CK en vivo tiene solo **6 técnicas** (cabeza T1190 trivial + cola larga sin soporte).

Definición del problema

Clasificación supervisada multiclase: $f : x \mapsto (y, p)$, con $y \in \{\text{benign, suspicious, malicious}\}$.
Representación x en tres niveles:

- campos tabulares del evento;
- mensaje crudo saneado;
- subgrafo de vecindario temporal (± 5 min).

Condicionantes: desbalance 245:1, distribución no estacionaria, humano en el bucle.

Preprocesamiento (P1–P5)

- **P1** Codificación categórica consciente del saneamiento (ausentes = nivel propio).
- **P2** Características temporales + partición temporal anti-leakage.
- **P3** Desbalance: muestreo estratificado + ponderación de clases (sin SMOTE sobre codificadas).
- **P4** Vectorización del texto del registro (TF-IDF 5k).
- **P5** Subgrafo de vecindario ± 5 min por host/credencial.

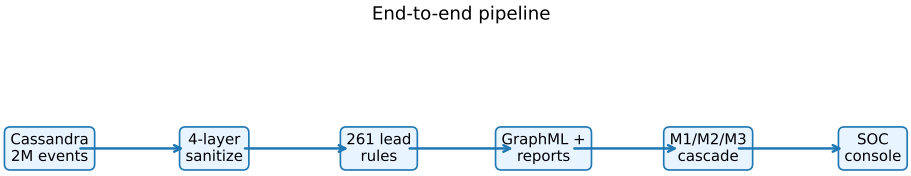
Cumple el requisito de al menos tres metodologías.

Modelos y complejidad

- **M1 tabular** (gradient boosting, ponderado): fit crece con árboles$\times$filas$\times$características; inferencia = recorrido raíz-hoja.
- **M2 texto**: concatena vectorización del mensaje + tabular; costo extra lineal en longitud del mensaje.
- **M3 grafo**: agrega vecindario en ventana y alimenta clasificador; costo por evento según tamaño del vecindario.

Cascada: M1 ordena la cola $\rightarrow$ M2 confirma con evidencia textual $\rightarrow$ M3 confirma con evidencia estructural.

Diagrama del proceso



De la captura y saneamiento, por las 261 reglas y el preprocesamiento, hasta la cascada de modelos y la consola del analista.

Protocolo de evaluación

- **Partición temporal**: entrenar con el pasado, evaluar con el futuro (corte = cuantil 80 de la marca de tiempo).
- **Métricas**: F1-macro y área bajo precisión-recuerdo por clase; recall malicioso a FPR fijo.
- La exactitud se reporta solo para mostrar que no es informativa bajo IR 245:1.
- Se documentarán semillas e infraestructura para permitir replicación.

Experimentos planeados

- **E1 (base tabular)**: M1 sobre campos. Hipótesis: recupera benign/suspicious, colapsa en malicious.
- **E2 (texto)**: M2 + mensaje. Hipótesis: eleva sustancialmente el recall malicioso.
- **E3 (estructura)**: M3 + vecindario. Hipótesis: iguala E2 por mecanismo distinto y refleja movimiento lateral.

Ablaciones: (i) codificación del texto (frecuencias vs contextual); (ii) radio de la ventana temporal.

Análisis previsto y despliegue

- Matriz de confusión normalizada por fila: hacia qué clase se desplaza el error minoritario.
- Explicitar el **compromiso**: recuperar la clase minoritaria vs preservar el nivel intermedio.
- **Despliegue**: FastAPI en :8000 (`/score`, `/queue`, `/incident`) + consola React en el mismo origen (Docker), con bitácora de feedback para re-entrenar.

Limitaciones y trabajo futuro

Limitaciones previstas

- Etiquetas de correlación automática, no de analista (oráculo imperfecto).
- Distribución desigual por tiempo y organización $\rightarrow$ generalización restringida.
- Análogo a CVSS: la etiqueta **depende del asignador** y su contexto.

Trabajo futuro

- Predicción multi-etiqueta de técnicas ATT&CK.
- Codificadores contextuales entrenables sobre los reportes de incidente.

Referencias (1/2)

- **Devign** — Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu. *Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks*. NeurIPS 2019. arXiv:1909.03496.
  Por qué: precedente del componente de grafo; demuestra que la estructura supera a los analizadores estáticos.
- **CVSS-BERT** — M. R. Shahid, H. Debar. *CVSS-BERT: Explainable NLP to Determine the Severity of a Computer Security Vulnerability from its Description*. IEEE ICMLA 2021. arXiv:2111.08510.
  Por qué: precedente del componente de texto; su saliency inspira nuestra explicabilidad.
- **Schwab & Gwerder** — *An Investigation into Vulnerability Databases*. Univ. de Berna, 2020.
  Por qué: evidencia que los puntajes divergen hasta 0.8 pts entre NVD y otros (y 1.3 por versión) $\rightarrow$ sustenta la limitación de las etiquetas.
- **LADC 2024** — *How Context Impacts Vulnerability Severity: An Analysis of Product-Specific CVSS Scores*. doi 10.1145/3697090.3697109.
  Por qué: NVD tiende a puntuar mayor que los CNA; el desacuerdo se concentra en sub-métricas dependientes del producto.
- **Le et al.** — *An Investigation into Inconsistency of Software Vulnerability Severity across Data Sources*. arXiv:2112.10356, 2021.
  Por qué: la inconsistencia entre fuentes degrada hasta 77 % la predicción aguas abajo.

Referencias (2/2)

- **NODLINK** — arXiv:2311.02331 (NDSS 2024). Detección de APT en línea sobre grafos de proveniencia.
- **KnowHow** — arXiv:2509.05698 (NDSS 2026). CTI + proveniencia, interpretable y con baja tasa de falsos positivos.
- **FLASH** — doi 10.1109/SP54263.2024.00139 (IEEE S&P 2024). GNN + semántica temporal.
- **threaTrace** — arXiv:2111.04333. GraphSAGE a nivel de nodo para amenazas en hosts.
- **GraphSAGE** — W. Hamilton et al., NeurIPS 2017. Aprendizaje inductivo en grafos grandes (base de M3).
- **TRAM** — MITRE Center for Threat-Informed Defense. `github.com/center-for-threat-informed-defense/tram`. Mapeador de reportes a ATT&CK + notebooks SciBERT.
- **TF-IDF vs transformers** — arXiv:2407.06753. TF-IDF gana con poco dato (P 75 %, F1 64 %) $\rightarrow$ justifica empezar por TF-IDF.
- **Focal Loss** — T.-Y. Lin et al., ICCV 2017. Pérdida para desbalance extremo.
- **Cost-sensitive IDS** — arXiv:0807.2043. Matriz de costos FN $>>$ FP $\rightarrow$ justifica no usar accuracy.
- **ATT&CK** — MITRE, *ATT&CK Design and Philosophy*, 2020. `attack.mitre.org`.
- **Dataset** — WitFoo Precinct6 v2.1.0, Apache-2.0. `huggingface.co/datasets/witfoo/precinct6-cybersecurity`.